

Fond de carte OpenStreetMap

Javions – étape 8

1. Introduction

Cette étape a pour but d’écrire les classes permettant de télécharger et d’afficher la carte sur laquelle les aéronefs apparaîtront.

2. Concepts

2.1. Fond de carte OpenStreetMap

Comme l’illustre la copie d’écran de la figure 1 ci-dessous, l’interface graphique de Javions montre une carte sur laquelle les aéronefs sont dessinés.

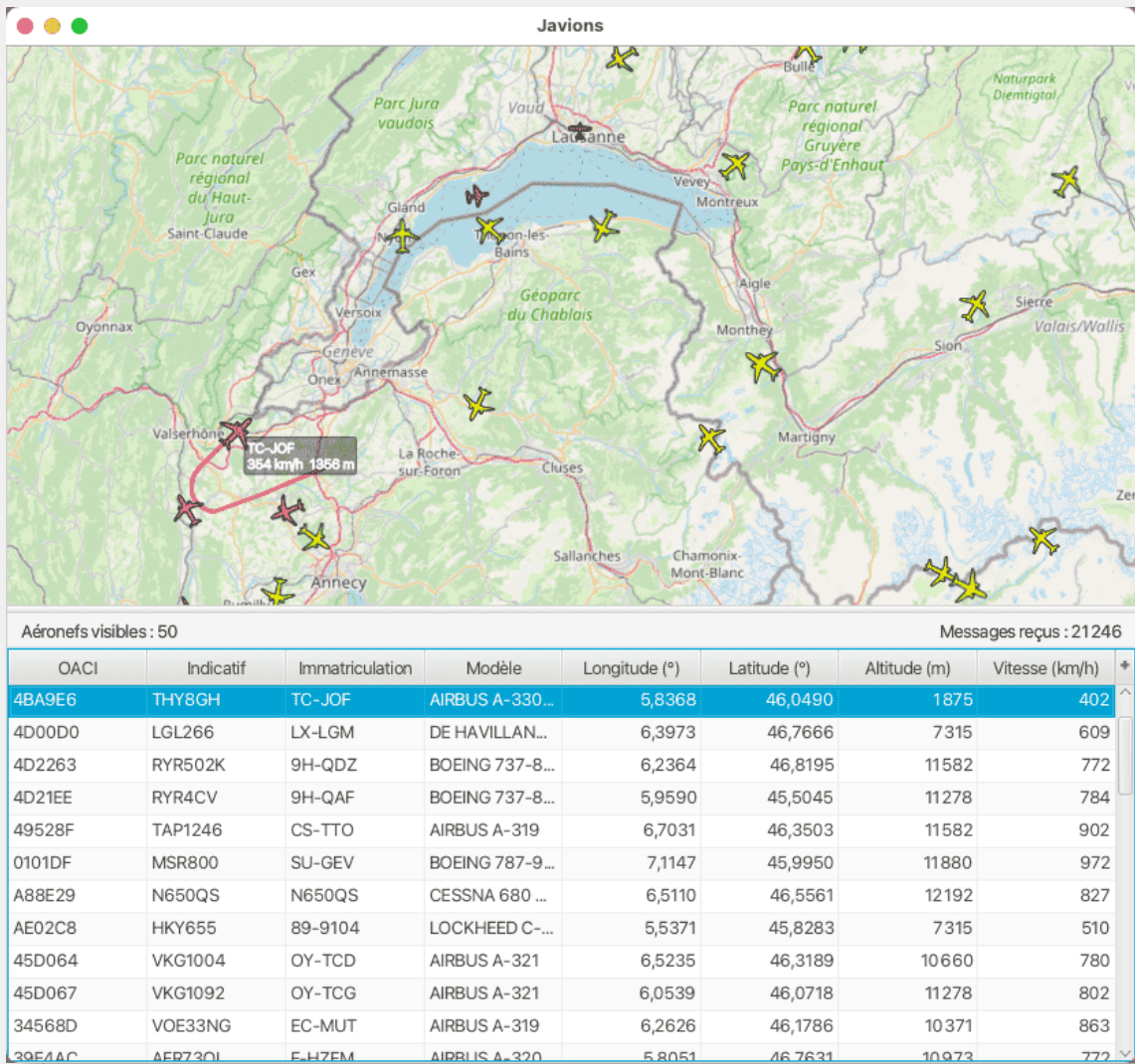


Figure 1 : L’interface graphique de Javions

L’image de la carte est celle d’OpenStreetMap, visible également sur [le site Web principal du projet](#). Afin de l’afficher dans l’interface, Javions la télécharge directement depuis les

serveurs mis à disposition par le projet OpenStreetMap.

L'interaction avec la carte s'effectue de la même manière qu'avec les systèmes cartographiques en ligne, c'est-à-dire que :

- le niveau de zoom – l'échelle – de la carte peut être changé au moyen de la molette de la souris ou du *touchpad*,
- la partie de la carte affichée peut être changée en maintenant pressé le bouton gauche de la souris en déplaçant cette dernière.

Ces deux types d'interactions sont visibles dans la vidéo ci-dessous.



Il faut noter que lorsque le niveau de zoom de la carte change, le point situé sous le curseur de la souris ne bouge pas. Cela est bien visible dans la vidéo ci-dessus, car le niveau de zoom y est changée à deux reprises, la première fois avec le curseur de la souris positionné dans les environs de l'EPFL, la seconde fois avec le curseur de la souris positionné aux alentours de Sauvabelin.

Le téléchargement et l'affichage de la carte est moins simple à effectuer qu'on pourrait le penser, car l'image représentant la carte n'est pas disponible en un seul morceau, mais est découpée en petites images carrées, nommées « tuiles », qui doivent être téléchargées séparément puis assemblées.

2.2. Tuiles OpenStreetMap

Comme nous l'avons vu à [l'étape 1](#), la carte OSM au niveau de zoom z est une image carrée de 2^{8+z} pixels de côté représentant la quasi-totalité de la planète – seules les latitudes au-delà de $\pm 85^\circ$ n'y figurent pas.

Aux niveaux de zoom faibles (0 à 4 environ), cette image est de taille raisonnable. Toutefois, aux niveaux de zoom élevés, elle devient énorme. Par exemple, au niveau 19, le nombre total de pixels dans l'image vaut :

$$(2^{8+19})^2 = 2^{54} = 18\,014\,398\,509\,481\,984 \approx 18 \cdot 10^{15}$$

soit plus de 18 pétapixels.

Sachant que chacun de ces pixels est représenté par une valeur de 32 bits (4 octets) contenant sa couleur empaquetée, l'image totale a une taille de 72 pétaoctets, ce qui est

conséquent. Pour donner un ordre de grandeur, la capacité du disque SSD d'un bon ordinateur portable actuel est d'environ 1 téraoctets, et il en faudrait donc 72000 pour stocker l'image de la carte OSM au niveau 19.

Il n'est donc clairement pas réaliste de représenter la carte OSM à un niveau de zoom donné au moyen d'une seule image. Au lieu de cela, l'image est découpée en petites images carrées nommées **tuiles** (*tiles*), et ces tuiles sont ensuite assemblées pour obtenir l'image correspondant à une zone du monde donnée.

Les tuiles OSM font 256 pixels de côté, ce qui se trouve être la taille de la carte entière au niveau de zoom 0. Cette carte-là est donc constituée d'une seule tuile. Au niveau de zoom 1, la carte est constituée de 4 tuiles ; au niveau de zoom 2, de 16 tuiles ; et ainsi de suite. Les tuiles sont indexées par deux index, (X, Y), la tuile du coin en haut à gauche ayant l'index (0, 0). Ces conventions sont illustrées dans l'image ci-dessous, qui montre les quatre tuiles OSM formant [la carte au niveau de zoom 1](#), avec leurs index.

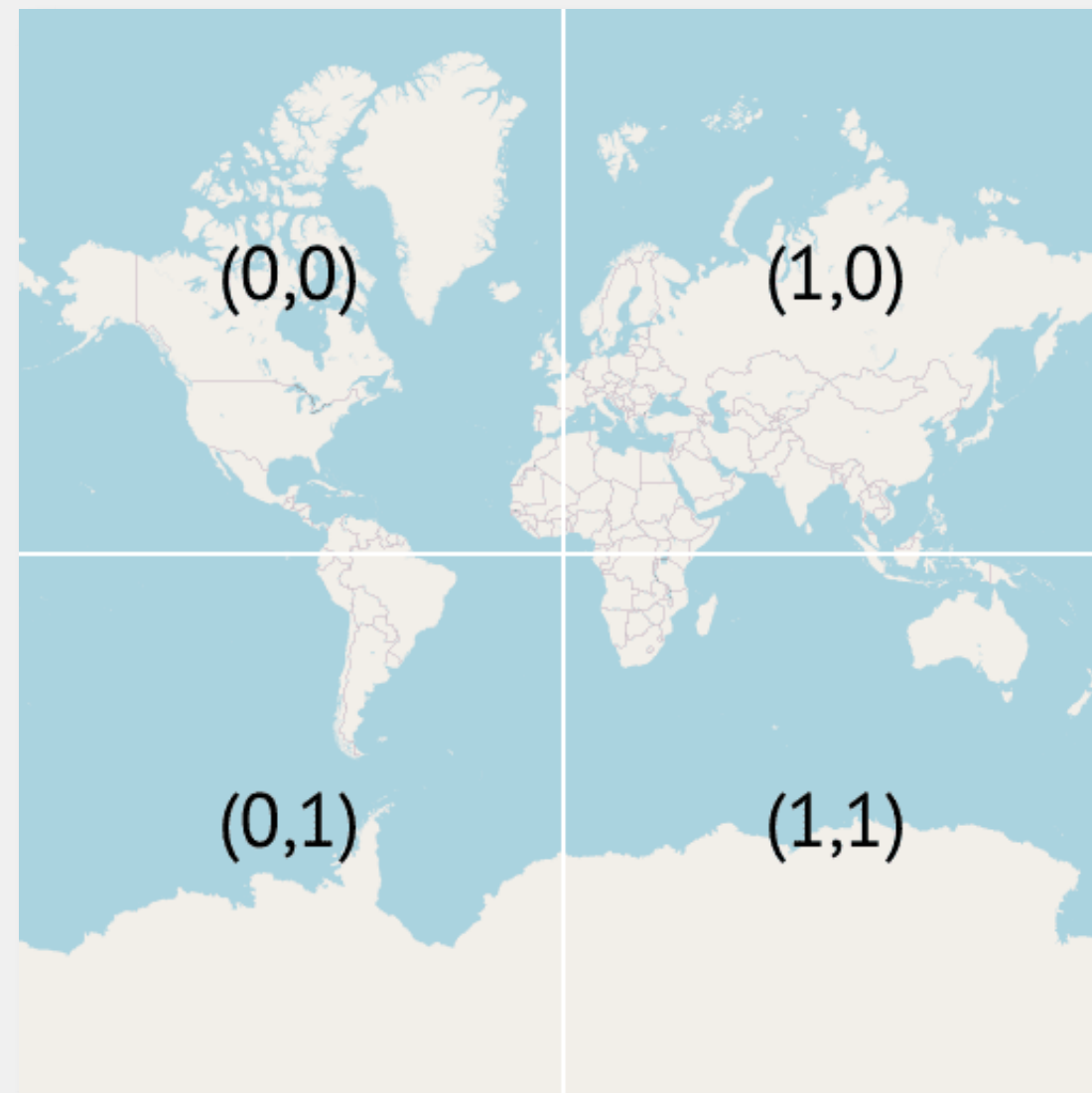


Figure 2 : Les quatre tuiles formant la carte OSM au niveau de zoom 1

À la §2.5 de [l'étape 1](#), nous avons décrit le système de coordonnées utilisé pour désigner la position d'un point sur l'image de la carte à un niveau de zoom z donné. Il existe bien entendu une correspondance directe entre les coordonnées (x_z, y_z) d'un point dans ce système de coordonnées et les index (X_z, Y_z) de la tuile le contenant, exprimée par les

équations suivantes :

$$X_z = \left\lfloor \frac{x_z}{256} \right\rfloor \quad Y_z = \left\lfloor \frac{y_z}{256} \right\rfloor$$

Par exemple, au niveau de zoom 17, le coin nord-ouest du *Learning Center* de l'EPFL a les coordonnées données par les formules de la §2.5 de l'étape 1 :

$$x = 2^{8+17} \left(\frac{0.114620}{2\pi} + \frac{1}{2} \right) \approx 17\,389\,327.34$$

$$y = 2^{8+17} \left(-\frac{\operatorname{arsinh}(\tan(0.811908))}{2\pi} + \frac{1}{2} \right) \approx 11\,867\,430.47$$

Il en découle que, au niveau de zoom 17, les index de la tuile le contenant sont :

$$X_{17} = \left\lfloor \frac{17\,389\,327}{256} \right\rfloor = 67\,927$$

$$Y_{17} = \left\lfloor \frac{11\,867\,430}{256} \right\rfloor = 46\,357$$

Cette tuile est visible ci-dessous.

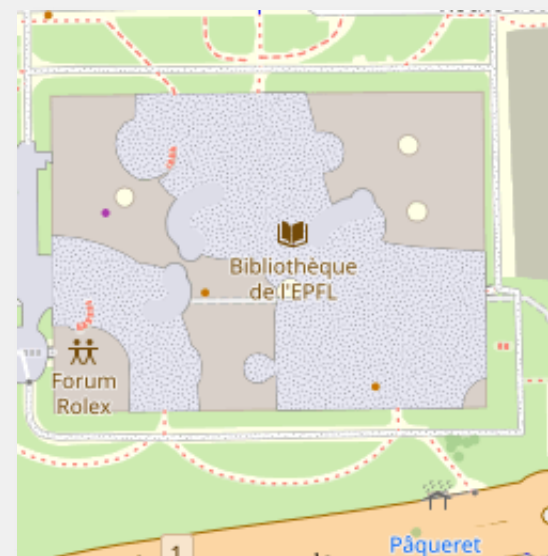
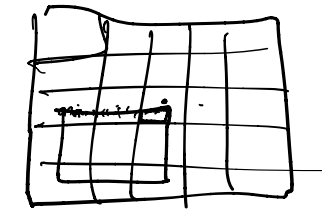


Figure 3 : La tuile OSM d'index (67927, 46357) au niveau de zoom 17

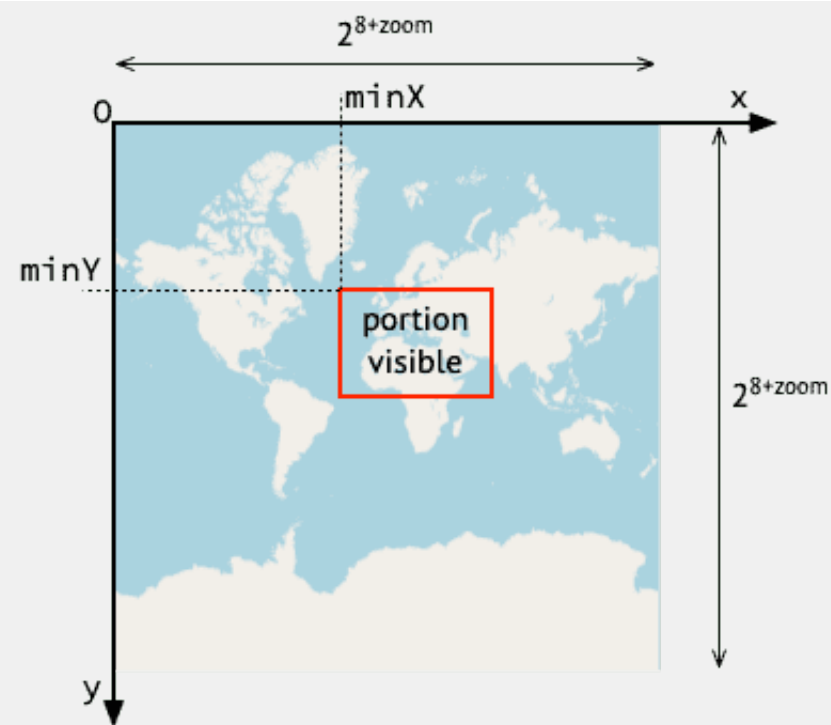
2.3. Portion visible de la carte

Lorsqu'un utilisateur interagit avec Javions, il ne voit généralement qu'une toute petite portion rectangulaire de l'image représentant la carte du monde entier. Cela est particulièrement vrai aux niveaux de zoom élevés.

Cette portion visible est illustrée dans l'image ci-dessous, où les coordonnées de son coin haut-gauche sont nommées $\min X$ et $\min Y$. Ces coordonnées sont exprimées dans le système de coordonnées de l'image au niveau de zoom courant.



0 0
(46 48) (47 48)



Bien entendu, le fait que la carte soit découpée en tuiles signifie que seules celles qui intersectent la portion visible de la carte doivent être téléchargées et dessinées.

2.4. Serveur de tuiles

Les tuiles OpenStreetMap sont stockées sur des ordinateurs gérés par le projet, que l'on nomme **serveurs de tuiles** (*tile servers*). Ces ordinateurs sont connectés à Internet et offrent la possibilité de télécharger les tuiles au moyen du protocole HTTPS – le protocole du Web.

Le serveur de tuiles OSM principal porte le nom `tile.openstreetmap.org`, et c'est lui qui fournit, entre autres, les tuiles visibles sur [le site OSM principal](#). Nous l'utiliserons également pour Javions. D'autres serveurs de tuile existent, qui donnent souvent accès à des tuiles dessinées dans un style différent de celui utilisé par le serveur principal. Ils sont répertoriés sur [la page Raster tile providers du Wiki OSM](#).

Pour obtenir l'image d'une tuile, il suffit de déterminer son URL¹ et d'effectuer une requête HTTPS au serveur de tuile pour obtenir l'image correspondante. L'URL d'une tuile est déterminée par son niveau de zoom et ses index, et a la forme suivante :

```
https://tile.openstreetmap.org/<zoom>/<X>/<Y>.png
```

où `<zoom>` représente le niveau de zoom, `<X>` l'index X de la tuile et `<Y>` son index Y. Par exemple, l'URL suivante :

```
https://tile.openstreetmap.org/17/67927/46357.png
```

permet d'obtenir l'image de la tuile d'index (67927, 46357) au niveau de zoom 17, présentée à l'image 3. Ces images sont fournies au format [PNG](#), un format d'image

fréquemment utilisé sur le Web.

2.5. Cache de tuiles

Grâce aux serveurs de tuiles, il est possible de télécharger très facilement n'importe quelle tuile d'une carte OSM. Ce téléchargement a toutefois un coût, puisqu'il implique le transfert, via Internet, de données situées sur un ordinateur distant.

Pour cette raison, un programme comme Javions doit impérativement limiter autant que possible le téléchargement direct de tuiles depuis le serveur. Pour ce faire, lors du premier téléchargement d'une tuile, il la stocke localement – en mémoire et/ou sur disque – afin d'éviter de devoir la télécharger à nouveau la prochaine fois qu'elle sera nécessaire.

L'emplacement dans lequel on stocke ainsi des ressources distantes afin de pouvoir y accéder plus rapidement lors de la prochaine utilisation s'appelle généralement un **cache** (ou **mémoire cache**, *cache memory*) en informatique.

Pour Javions, nous stockerons les tuiles OSM dans deux caches de tuiles : un cache mémoire de relativement petite taille, et un cache disque de très grande taille.

2.5.1. Cache mémoire

Un cache mémoire stocke, directement dans la mémoire du programme, des valeurs qui sont chères à obtenir. Dans le cas de Javions, le cache de tuile stocke, sous la forme d'objets Java représentant des images, un certain nombre de tuiles obtenues à l'origine depuis le serveur OSM.

Ce cache a une taille limitée, car la mémoire à disposition d'un programme l'est également. Sachant qu'une tuile OSM fait 256 pixels de côté, la mémoire nécessaire au stockage de ses pixels, en octets, est de :

$$256^2 \times 4 = 262\,144$$

donc environ 260 kilooctets. En gardant 100 tuiles en mémoire, on utilise donc environ 26 mégaoctets, ce qui semble être une limite raisonnable.

2.5.2. Cache disque

Un cache mémoire a l'avantage d'être extrêmement rapide, mais l'inconvénient d'être limité en capacité. Il est donc fréquemment judicieux d'utiliser, en plus d'un cache mémoire de faible capacité, un cache disque de plus grande capacité. Comme son nom l'indique, ce cache stocke ses données sur le « disque » – disque dur ou SSD – de l'ordinateur, dans des fichiers.

Le cache disque de Javions stocke les images des tuiles sous la forme de fichiers image au format [PNG](#), puisque c'est celui qui est utilisé par les serveurs de tuile. Les tuiles sont

stockées dans des dossiers organisés ainsi :

- au niveau supérieur de la hiérarchie se trouve un dossier par niveau de zoom,
- chacun de ces dossiers contient un sous-dossier par index X de tuile,
- chacun de ces sous-dossiers contient un fichier PNG par tuile.

Par exemple, sachant que, au niveau de zoom 17, la tuile contenant le *Learning Center* a les index (67927, 46357), le cache disque stocke son image dans un fichier nommé 46357.png qui se trouve dans un dossier nommé 67927 situé lui-même dans un dossier nommé 17. Cette hiérarchie peut être représentée schématiquement ainsi :

```
17
└─ 67927
   └─ 46357.png
```

3. Mise en œuvre Java

3.1. Enregistrement `TileManager.TileId`

L'enregistrement `TileId`, imbriqué dans la classe `TileManager` décrite à la section suivante, représente l'identité d'une tuile OSM. Il possède trois attributs, qui sont :

- le niveau de zoom de la tuile, nommé p. ex. `zoom`,
- l'index X de la tuile, nommé p. ex. `x`,
- l'index Y de la tuile, nommé p. ex. `y`.

`TileId` offre une méthode publique et statique, nommée p. ex. `isValid`, prenant en argument ces trois attributs (zoom et index X/Y) et retournant vrai si – et seulement si – ils constituent une identité de tuile valide.

3.2. Classe `TileManager`

La classe `TileManager` du sous-paquetage `gui`, publique et finale, représente un gestionnaire de tuiles OSM. Son rôle est d'obtenir les tuiles depuis un serveur de tuile et de les stocker dans un cache mémoire et dans un cache disque.

Son constructeur prend en arguments :

1. le chemin d'accès au dossier contenant le cache disque, de type `Path`,
2. le nom du serveur de tuile (p.ex. `tile.openstreetmap.org`).

La seule méthode publique offerte par `TileManager`, nommée p. ex. `imageForTileAt`, prend en argument l'identité d'une tuile (de type `TileId`) et retourne son image (de type `Image` de la bibliothèque JavaFX).

L'image est cherchée tout d'abord dans le cache mémoire, et si elle s'y trouve, est

simplement retournée. Sinon, elle est cherchée dans le cache disque, et si elle s'y trouve, elle est chargée, placée dans le cache mémoire et retournée. Sinon, elle est obtenue depuis le serveur de tuiles, placée dans le cache disque, placée dans le cache mémoire et enfin retournée.

3.2.1. Conseils de programmation

1. Limitation de la taille du cache mémoire

Le cache mémoire doit contenir un maximum de 100 images. Lorsqu'il est plein, il faut donc supprimer l'une des images qu'il contient, mais pas n'importe laquelle ! Il semble en effet plus judicieux de choisir celle utilisée le moins récemment (*least-recently used* ou LRU en anglais), car c'est probablement la moins utile de toutes.

La bibliothèque Java offre la classe `LinkedHashMap`, qui est une table associative similaire à `HashMap` mais qui offre un constructeur permettant de demander à ce que les éléments soient parcourus du moins récemment accédé au plus récemment accédé. Utilisez-le pour facilement trouver l'élément du cache à supprimer lorsqu'il est plein.

2. Téléchargement et stockage d'une tuile

Il est très facile d'obtenir un flot d'entrée fournissant les données d'une tuile grâce à la classe `URLConnection`, comme l'illustre l'extrait de programme suivant :

```
URL u = new URL(
    "https://tile.openstreetmap.org/17/67927/46357.png");
URLConnection c = u.openConnection();
c.setRequestProperty("User-Agent", "Javions");
InputStream i = c.getInputStream();
```

Bien entendu, il est impératif de fermer le flot retourné par `getInputStream`, ce qui n'a pas été fait ci-dessus. Pour cela, utilisez la notation *try-with-resource*.

Un petit problème se pose au téléchargement d'une tuile, car le flot retourné par `getInputStream` doit, en quelque sorte, être utilisé deux fois : une fois pour écrire le contenu de la tuile dans le cache disque, et une fois pour construire l'image. Or un flot ne peut pas être réutilisé ainsi, car dès que ses données ont été consommées, il est vide.

Une solution assez efficace à ce problème consiste à utiliser la méthode `readAllBytes` pour obtenir un tableau d'octets contenant les données de l'image, puis à utiliser ce tableau deux fois : une fois pour écrire les données dans le cache disque – au moyen de la méthode `write` d'un flot de sortie – et une fois pour créer un flot de type `ByteArrayInputStream` à passer au constructeur de `Image` prenant un flot d'entrée en argument.

Notez pour terminer que certaines méthodes de la classe `Files` pourraient vous être utiles pour gérer le cache disque, p. ex. `exists` et `createDirectories`.

3.3. Classe `MapParameters`

La classe `MapParameters` du sous-paquetage `gui`, publique et finale, représente les paramètres de la portion de la carte visible dans l'interface graphique. Elle possède trois propriétés JavaFX, qui sont :

- une propriété, nommée p. ex. `zoom`, contenant une valeur de type `int` représentant le niveau de zoom,
- une propriété, nommée p. ex. `minX`, contenant une valeur de type `double` représentant la coordonnée `x` du coin haut-gauche de la portion visible de la carte,
- une propriété, nommée p. ex. `minY`, contenant une valeur de type `double` représentant la coordonnée `y` du coin haut-gauche de la portion visible de la carte.

Notez que la largeur et la hauteur de la portion visible de la carte ne sont pas stockées dans des propriétés de `MapParameters`, car elles sont déterminées par les dimensions de la fenêtre dans laquelle la carte est affichée.

Le niveau de zoom est limité à la plage allant de 6 (inclus) à 19 (inclus). Cette plage convient bien à ce projet, car la portée limitée de la radio signifie qu'il n'est pas vraiment utile d'afficher la carte à un très petit niveau de zoom.

Les propriétés `minX` et `minY` sont exprimées dans le système de coordonnées Web Mercator de l'image au niveau de zoom stocké dans la propriété correspondante. Pour mémoire, l'origine de ce système de coordonnées est le coin haut-gauche de l'image représentant la carte de la Terre entière, et l'axe des abscisses est dirigé vers la droite, celui des ordonnées vers le bas.

Par exemple, pour la carte affichée dans l'interface de la figure 1, ces trois propriétés contiennent : 9 (`zoom`), 67522 (`minX`) et 46287 (`minY`). Le point correspondant au coin haut-gauche est indiqué par le marqueur bleu sur [cette carte](#), et on peut se convaincre visuellement qu'il se trouve bien à la position correspondant au coin haut-gauche de la carte de la figure 1.

Le constructeur de `MapParameters` prend en arguments les valeurs initiales de ces trois propriétés, et lève une exception si le niveau de zoom donné n'est pas dans les limites susmentionnées.

En plus de ce constructeur, `MapParameters` offre deux méthodes publiques pour chacune de ses trois propriétés, la première retournant la propriété elle-même, en lecture seule, et la seconde retournant sa valeur.

Les propriétés sont en lectures seules, car elles sont destinées à n'être modifiées qu'au moyen de l'une des deux méthodes suivantes, elles aussi publiques :

- une méthode, nommée p. ex. `scroll`, qui prend en argument les composantes `x` et `y` d'un vecteur, et translate le coin haut-gauche de la portion de carte affichée de ce vecteur,
- une méthode, nommée p. ex. `changeZoomLevel`, qui prend en argument une différence de niveau de zoom – positive, négative ou nulle – et l'ajoute au niveau de zoom actuel, en garantissant toutefois qu'il reste dans les limites susmentionnées.

La méthode `changeZoomLevel` doit conserver la position du coin haut-gauche de la portion visible de la carte, ce qui implique qu'elle doit adapter les valeurs des propriétés `minX` et `minY` pour tenir compte du fait que le système de coordonnées change lorsque le niveau de zoom change.

utiliser
clamp
de Math2

3.3.1. Conseils de programmation

Une méthode de la classe `Math2`, écrite lors de l'étape 1, peut être utile pour garantir que le niveau de zoom reste dans les limites prescrites.

3.4. Classe `BaseMapController`

La classe `BaseMapController` du sous-paquetage `gui`, publique et finale, gère l'affichage et l'interaction avec le fond de carte. Elle possède un constructeur public qui prend en arguments :

- le gestionnaire de tuiles à utiliser pour obtenir les tuiles de la carte,
- les paramètres de la portion visible de la carte.

En plus de ce constructeur public, `BaseMapController` offre deux méthodes publiques :

- une méthode, nommée p. ex. `pane`, qui retourne le panneau JavaFX affichant le fond de carte,
- une méthode, nommée p. ex. `centerOn`, qui prend en argument un point à la surface de la Terre, de type `GeoPos`, et déplace la portion visible de la carte afin qu'elle soit centrée en ce point.

La méthode `centerOn` sera utilisée plus tard dans le projet pour centrer la carte sur un aéronef donné.

3.4.1. Conseils de programmation

3 4 □

1. Hiérarchie JavaFX

□ 7 8

La carte est dessinée sur une instance de `Canvas` – la classe JavaFX représentant un « canevas », c.-à-d. une surface sur laquelle il est possible de dessiner – placée dans un panneau de type `Pane`.

La raison pour laquelle il est nécessaire de placer le canevas dans un panneau est que, contrairement au canevas, le panneau est redimensionné automatiquement lorsqu'il est placé dans certains composants. Cela est utile pour garantir que le panneau (et donc la carte) occupe la totalité de la fenêtre de l'interface.

Sachant que le canevas n'est pas redimensionné automatiquement, il faut utiliser des liens JavaFX (*bindings*) pour faire en sorte que sa largeur et sa hauteur soient toujours égales à celles du panneau. Pour la largeur, cela peut se faire ainsi :

```
canvas.widthProperty().bind(pane.widthProperty());
```

2. Dessin de la carte

Pour dessiner sur un canevas, il faut obtenir ce que l'on nomme son **contexte graphique** (*graphics context*) au moyen de la méthode `getGraphicsContext2D`. Ce contexte, de type `GraphicsContext`, possède de nombreuses méthodes permettant de définir les paramètres de dessin (couleur du trait, épaisseur, etc.) et de dessiner.

Pour dessiner la carte, une fois le contexte graphique obtenu, on peut utiliser sa méthode `drawImage` pour dessiner chacune des tuiles visible au moins partiellement. Ces tuiles sont obtenues du gestionnaire de tuiles, et si ce dernier lève une exception (de type `IOException`), la tuile correspondante n'est simplement pas dessinée.

Il faut noter que, comme le coin haut-gauche de la portion de la carte visible ne correspond généralement pas au coin haut-gauche d'une tuile, les coordonnées passées à `drawImage` sont souvent négatives pour les tuiles situées sur les bords haut et gauche de la fenêtre. Cela ne pose toutefois pas de problème, JavaFX se chargeant de ne dessiner que la partie visible des images.

3. Redessin efficace

La carte doit être redessinée à chaque changement des paramètres de la carte affichée (niveau de zoom ou coin haut-gauche), ainsi qu'à chaque changement des dimensions du canevas.

Le dessin de la carte étant relativement cher, il est important de le retarder autant que possible. Par exemple, si l'utilisateur redimensionne rapidement la fenêtre, il faudrait éviter de redessiner la totalité de la carte au moindre changement.

Pour cela, nous vous proposons une technique qui consiste à retarder le redessin jusqu'au prochain « battement » (*pulse*) JavaFX. Sans rentrer dans les détails, JavaFX met généralement à jour le dessin de l'interface 60 fois par seconde, si cela est nécessaire. Chacune de ces mises à jour est nommée un « battement ». En retardant le redessin de la carte au prochain battement, on garantit qu'elle n'est pas redessinée plus de 60 fois par seconde.

Retarder ainsi le redessin n'est pas très facile et utilise des notions avancées de JavaFX, donc nous vous offrons ici une solution «clefs en main». Il vous faut dans un premier temps ajouter à votre classe un attribut booléen nommé `redrawNeeded`, qui ne sera vrai que si un redessin de la carte est nécessaire. Ensuite, il vous faut y ajouter une méthode privée nommée `redrawIfNeeded` effectuant le redessin si et seulement si cet attribut est vrai :

```
private void redrawIfNeeded() {  
    if (!redrawNeeded) return;  
    redrawNeeded = false;  
  
    // ... à faire : dessin de la carte  
}
```

Ensuite, il faut vous assurer que JavaFX appelle bien cette méthode à chaque battement, ce qui se fait au moyen du code ci-dessous, à ajouter au constructeur de votre classe :

```
canvas.sceneProperty().addListener((p, oldS, newS) -> {  
    assert oldS == null;  
    newS.addPreLayoutPulseListener(this::redrawIfNeeded);  
});
```

Finalement, il faut ajouter une méthode privée à votre classe permettant de demander un redessin au prochain battement. Cela se fait en mettant à vrai l'attribut `redrawNeeded`, et en forçant JavaFX à effectuer le prochain battement, même si de son point de vue cela n'est pas nécessaire :

```
private void redrawOnNextPulse() {  
    redrawNeeded = true;  
    Platform.requestNextPulse();  
}
```

Une fois tout cela en place, il ne vous reste plus qu'à appeler la méthode `redrawOnNextPulse` lorsqu'un redessin est nécessaire.

4. Gestion des événements

`BaseMapController` doit détecter et gérer deux types d'événements :

1. l'utilisation de la molette de la souris ou du *touchpad*, qui permettent de changer le niveau de zoom de la carte,
2. le déplacement de la souris lorsque le bouton gauche est maintenu pressé, qui permet de faire glisser la carte.

Pour gérer ces événements, il faut installer des gestionnaires sur le panneau contenant la carte, au moyen des méthodes `setOnScroll` (changement du niveau de zoom), `setOnMousePressed`, `...Dragged`, `...Released` (glissement de la carte). Il faut noter que ces gestionnaires s'écrivent beaucoup plus simplement au moyen de lambdas. Si vous avez de l'avance et que vous n'avez pas encore lu [les notes de cours sur ce sujet](#), nous vous conseillons de le faire avant d'essayer de les programmer.

Comme tous les gestionnaires d'événements JavaFX, ceux qu'on installe au moyen de ces méthodes reçoivent en argument un événement, de type `MouseEvent` pour tous les gestionnaires susmentionnés sauf celui passé à `setOnScroll`, qui reçoit un événement de type `ScrollEvent`. Ces événements possèdent plusieurs méthodes utiles parmi lesquelles :

- `getX()` et `getY()`, qui permettent de connaître la position du curseur de la souris,
- `getDeltaY()` (`ScrollEvent` uniquement), qui permet de savoir dans quelle direction la molette de la souris a été tournée.

Afin que le changement de niveau de zoom puisse se faire de manière agréable aussi bien avec la molette de la souris qu'avec un *touchpad*, il faut écrire du code non trivial qui d'une part ne prend en compte que le signe de la valeur retournée par `getDeltaY`, et d'autre part limite la fréquence à laquelle les changements de zoom peuvent avoir lieu. Pour faciliter votre travail, ce code vous est donné ci-dessous, et vous pouvez directement l'inclure dans votre projet :

```
LongProperty minScrollTime = new SimpleLongProperty();
pane.setOnScroll(e -> {
    int zoomDelta = (int) Math.signum(e.getDeltaY());
    if (zoomDelta == 0) return;

    long currentTime = System.currentTimeMillis();
    if (currentTime < minScrollTime.get()) return;
    minScrollTime.set(currentTime + 200);

    // ... à faire : appeler les méthodes de MapParameters
});
```

Comme cela a été mentionné plus haut, le point se trouvant sous le pointeur de la souris ne doit pas bouger lorsque le niveau de zoom change. Sachant que la méthode `changeZoom` de `MapParameters` préserve la position du coin haut-gauche de la portion de carte affichée à l'écran, la manière la plus simple de procéder consiste à effectuer deux translations du coin haut-gauche de la portion visible de la carte : la première avant le changement de niveau de zoom, pour placer ce coin sous le pointeur de la souris, la seconde après le changement de

niveau de zoom, pour annuler l'effet de la première.

Les trois gestionnaires d'événements gérant le glissement de la carte doivent déterminer de combien d'unités la souris s'est déplacée le long des deux axes. Un moyen simple de faire cela est de stocker dans une propriété JavaFX la position à laquelle se trouvait la souris lors de l'événement précédent, représentée par une valeur de type `Point2D`. Cette classe possède des méthodes (`add`, `subtract`, etc.) permettant de facilement faire les calculs nécessaires.

En plus de ces gestionnaires d'événements, `BaseMapController` doit également mettre en place des auditeurs JavaFX qui détectent les situations dans lesquelles le fond de carte doit être redessiné, et appeler `redrawOnNextPulse` dans ce cas. Cela se fait facilement en ajoutant un auditeur détectant les changements des paramètres du fond de carte et des dimensions du canevas – c.-à-d. ses propriétés `width` et `height`.

3.5. Tests

Comme d'habitude dans la seconde partie du projet, aucun test ne vous est fourni. Nous vous donnons néanmoins ci-après quelques conseils pour tester certaines parties de cette étape.

3.5.1. Classe `TileManager`

Pour tester votre gestionnaire de tuiles, vous pouvez l'utiliser pour obtenir la tuile contenant le *Learning Center* et vérifier qu'elle est bien stockée dans le cache disque.

Pour ce faire, il vous faut écrire une toute petite application JavaFX, ressemblant à ceci :

```
public final class TestTileManager extends Application {
    public static void main(String[] args) { launch(args); }

    @Override
    public void start(Stage primaryStage) throws Exception {
        new TileManager(Path.of("tile-cache"),
                        "tile.openstreetmap.org")
            .imageForTileAt(new TileId(17, 67927, 46357));
        Platform.exit();
    }
}
```

Une fois cette application exécutée, vous devriez avoir, dans le dossier de votre projet, un dossier nommé `tile-cache` contenant un sous-dossier nommé `17`, contenant un sous-dossier nommé `67927`, contenant un fichier nommé `46357.png`. En ouvrant ce fichier, vous devriez voir l'image de la figure 3.

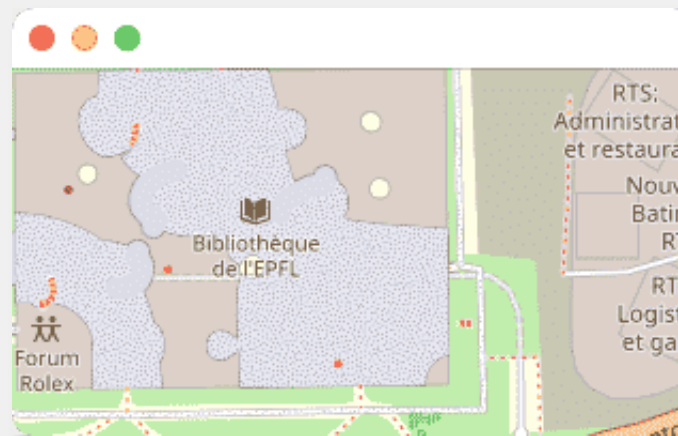
3.5.2. Classe BaseMapController

Après vous être assurés que votre gestionnaire de tuile fonctionne correctement, vous pouvez tester le contrôleur du fond de carte au moyen d'une petite application JavaFX similaire à celle ci-dessous.

```
public final class TestBaseMapController extends Application {
    public static void main(String[] args) { launch(args); }

    @Override
    public void start(Stage primaryStage) {
        Path tileCache = Path.of("tile-cache");
        TileManager tm =
            new TileManager(tileCache, "tile.openstreetmap.org");
        MapParameters mp =
            new MapParameters(17, 17_389_327, 11_867_430);
        BaseMapController bmc = new BaseMapController(tm, mp);
        BorderPane root = new BorderPane(bmc.pane());
        primaryStage.setScene(new Scene(root));
        primaryStage.show();
    }
}
```

En la lançant, vous devriez voir apparaître une fenêtre similaire à celle ci-dessous, et vous devriez de plus pouvoir naviguer dans la carte au moyen de la souris ou du *touchpad*, comme illustré dans la vidéo de la §2.1.



4. Résumé

Pour cette étape, vous devez :

- écrire les classes `TileManager`, `TileId`, `MapParameters` et `BaseMapController` selon les indications données ci-dessus,
- tester votre code,
- documenter la totalité des entités publiques que vous avez définies.

Aucun rendu n'est à faire pour cette étape avant le rendu final. N'oubliez pas de faire régulièrement des copies de sauvegarde de votre travail en suivant [nos indications à ce sujet](#).

Notes de bas de page

¹ Une [URL](#), ou adresse Web, est une chaîne de caractères identifiant une ressource (page, image, etc.) sur le Web. Par exemple, l'URL <https://www.epfl.ch/> identifie la page principale du site de l'EPFL, <https://cs108.epfl.ch/> identifie la page principale de ce cours, etc.